

Blue Eye - AI Voice Reception Agent

Comprehensive System Architecture & Technical Interview Guide

1. Executive Summary & Core Value Proposition

Blue Eye is a full-stack, two-way conversational AI Voice Receptionist application. It acts as a 24/7 intelligent front-desk agent that greets incoming callers, transcribes spoken queries, accurately extracts key details (caller name, caller intent, callback phone number), persists the conversation in a structured SQLite database with Indian Standard Time (IST) timestamps, and synthesizes an immediate, personalized audio response that automatically plays back in the caller's browser.

2. Key Architectural Components

- Frontend / UI Layer:** Streamlit web application (app.py) providing live microphone recording (st.audio_input), audio file browsing, audio playback, metrics, and call logs.
- Speech-to-Text (STT):** Sarvam AI Saaras v3 model - highly tuned for Indian accents and multilingual nuances; converts audio to verbatim text.
- Agent Reasoning (LLM):** Sarvam AI sarvam-105b Chat Completions API with 3-tier fallback architecture to extract structured JSON (caller_name, intent, callback_number, spoken_response).
- Text-to-Speech (TTS):** Sarvam AI Bulbul v3 model - converts the generated receptionist script into high-fidelity, natural Indian voice audio.
- Database Layer:** SQLite relational database (calls.db via database.py) with automated column migration, indexing, and IST timestamping.
- One-Click Launcher:** start_blue_eye.bat Windows launcher paired with a Desktop shortcut for instant startup without terminal commands.

3. End-to-End System Workflow

- Caller Initiation: Caller either clicks the live microphone or uploads an audio file.
- Auto Greeting: Blue Eye automatically plays a professional welcome greeting prompting for details.
- STT Transcription: Audio bytes are transmitted to Sarvam Saaras v3, generating verbatim English text.
- LLM Extraction: Prompt engineering guides sarvam-105b to extract exact details with strict zero-hallucination rules.
- Audio Synthesis: The receptionist's personalized reply is synthesized into WAV bytes via Bulbul v3.
- Autoplay Response: Browser automatically plays the agent's voice response using embedded HTML5 audio.
- Database Persistence: All records are saved to calls.db, populating the live CRM table with 1-click WhatsApp callbacks.

4. Project Directory & File Layout

- app.py:** Streamlit UI application, handling audio inputs, state, autoplay, and CRM records.
- agent.py:** LLM prompt design, JSON parser, and 3-layer resilient fallback engine.
- speech_to_text.py:** REST client integration for Sarvam Saaras v3 STT.
- text_to_speech.py:** REST client integration for Sarvam Bulbul v3 TTS with persona support.

- **database.py:** SQLite schema initialisation, migration, and CRUD queries.
- **generate_audio.py:** Offline utility for generating pre-recorded sample call scenarios.
- **pipeline_test.py:** End-to-end CLI integration test script verifying all 3 call scenarios.
- **start_blue_eye.bat:** Automated port-checking and browser launch script.
- **audio/:** Directory holding sample WAV files (Rahul, Priya, Arjun) and initial greetings.
- **calls.db:** Embedded SQLite storage containing logged call history.

5. Critical Engineering Challenges & Solutions

Challenge 1: LLM Truncated / Reasoning-Only Outputs

Reasoning models like sarvam-105b sometimes return output inside 'reasoning_content' rather than 'content', or get cut off mid-stream. We engineered a 3-layer recovery mechanism: first parsing content, then falling back to searching reasoning traces for valid JSON, and finally running keyword/regex heuristics to guarantee zero UI crashes.

Challenge 2: Seamless Autoplay Without Manual Downloads

Standard audio elements require manual user clicks or downloading. We solved this by generating Base64-encoded audio embedded directly into an HTML5 <audio autoplay> tag, delivering a natural phone call experience.

Challenge 3: Accurate Timezone Handling

Server defaults stored UTC timestamps, causing confusion for local office staff. We explicitly configured `datetime.now(timezone(timedelta(hours=5, minutes=30)))` to ensure exact Indian Standard Time (IST) logging.

6. High-Frequency Technical Interview Questions

Q1: Walk me through the architecture of your Voice Receptionist.

Answer: It follows an event-driven 5-stage pipeline: Audio Capture (Streamlit) -> STT (Sarvam Saaras v3) -> Information Extraction & Agent Reasoning (Sarvam 105B) -> Voice Synthesis (Sarvam Bulbul v3) -> Relational Storage (SQLite). The system is fully decoupled into modular Python services.

Q2: Why did you pick Sarvam AI instead of OpenAI Whisper/GPT-4?

Answer: Sarvam AI is specifically trained and optimized for Indian accents, speech cadences, and code-mixed vocabulary. During testing, phone numbers spoken with Indian accents were captured with far higher fidelity, and it provides an integrated suite covering STT, Chat LLM, and TTS with Indian personas (e.g. Kavya, Shubh).

Q3: How do you prevent LLM hallucinations on extracted phone numbers or names?

Answer: In agent.py, we implement strict zero-shot system prompts instructing the model to assign 'null' if data is absent. Furthermore, we sanitize callback numbers using regex (`r'\D'`) and cross-verify with phone number patterns (10 digits starting 6-9).

Q4: How does this project scale to handle thousands of concurrent calls in enterprise?

Answer: In production: 1) Replace SQLite with PostgreSQL or managed cloud SQL. 2) Use Sarvam's Streaming/WebSocket or Batch APIs. 3) Integrate Twilio/SIP telephony webhooks instead of browser mic. 4) Run the backend services inside containerized Docker/Kubernetes pods.